

ADR-0004 — Transport: HTTP/3 (QUIC) + Brotli with HTTP/2 Fallback

Field	Value
ADR	ADR-0004
Title	Transport & compression strategy (HTTP/3 (QUIC) primary, HTTP/2 + gzip fallback, Brotli content compression)
Status	Proposed
Created	2026-06-07
Last modified	2026-06-07
Revision	2
Fixed-summary	Rev 2: corrected master pointers (transport mandate §3; TLS 1.3 §6), re-attributed FILE_HASH/METADATA_HASH to aosp-update-engine §7 / android-update-engine-api.md §6, softened download-resume claims to UNVERIFIED, and added a §11.4.123 (spikes-as-rock-solid-proof) compliance nod.
Author	Lead architect (research synthesis)
Decides	Transport protocol, negotiation/fallback, content compression, and OTA-artifact download/resume strategy for the Helix control plane and Android client.
Anchors contradiction	additions_synthesis.md §5 C4 (REST+gRPC drafts vs mandated Gin + REST-primary + HTTP/3→HTTP/2 + Brotli).
Honors LOCKED	D2 native Android A/B + custom Go control plane; D6 mandated stack (Go + Gin + Brotli + HTTP/3(QUIC)→HTTP/2, REST primary); research decides wrap target (not re-opened here).
Sources	ota_landscape_report.md; stacks/aosp-update-engine.md; stacks/rauc.md; stacks/eclipse-hawkbit.md; stacks/ostree.md;

Field	Value
Anti-bluff	additions_synthesis.md; 00-master/2026-06-07-helix-ota-design.md. Every claim traces to a cited source note or the master design; items the notes flagged are carried forward as UNVERIFIED. This ADR introduces no transport facts absent from the underlying corpus, except where explicitly labelled UNVERIFIED (new, needs spike).

Table of contents

1. Context
2. Decision drivers
3. Options considered
 - 3.1 Transport protocol
 - 3.2 Content compression
 - 3.3 Artifact download & resumability
4. Decision
5. Consequences
 - 5.1 Positive
 - 5.2 Negative / costs
6. Open / UNVERIFIED items to close
7. Status
8. Compliance notes (HelixConstitution)

1. Context

Helix OTA's transport stack is **mandated, not open**: the operator-locked stack (D6) is *Go + Gin + Brotli + HTTP/3 (QUIC) → HTTP/2 fallback, REST primary*, delivered via the `vasic-digital/http3` submodule as a drop-in `net/http.Handler` with automatic fallback to HTTP/2 + standard compression, and Brotli content compression negotiating down to gzip for older clients [00-master §3, §4]. This ADR does **not** re-decide whether to use HTTP/3 or Brotli — that is locked. It decides **how** the transport is composed and, critically, **distinguishes two traffic classes with opposite requirements**:

- **Control-plane traffic** (REST /api/v1: poll/check-in, rollout assignment, telemetry ingest, dashboard) — many small JSON request/response pairs from up to millions of devices [00-master §1 scalability guarantee]. This class benefits from HTTP/3's connection-migration + head-of-line-blocking elimination and from Brotli/gzip text compression.
- **Artifact traffic** (the OTA `payload.bin` download) — large, opaque, **already-compressed** binary blobs that the device's `update_engine` byte-range-fetches by `offset/size` from an uncompressed (ZIP_STORED) ZIP [`aosp-update-engine`]

§6, §7]. This class has near-zero gain from generic content compression and a **hard requirement** for HTTP Range support.

Conflating these two classes is the central transport risk. The reconciliation already ruled (C4) that the mandated stack wins over the drafts' REST+gRPC proposals:

REST is the compatibility surface; gRPC is optional/internal only

[additions_synthesis.md §5 C4].

Device-network realities the corpus flags: device-pull streaming over plain HTTP(S) Range Requests “fits a custom Go control plane far better than engines that demand their own server protocol” [rauc §5], but “Brotli/HTTP3 specifics would need spike validation ... Range-request semantics over HTTP/2/3 should hold but is unverified for Helix’s exact stack” [rauc §5]. There is **no Google-mandated OTA server protocol**; the device side only needs a payload + metadata over HTTPS [aosp-update-engine §7].

2. Decision drivers

Driver	Source
Mandated stack is non-negotiable (HTTP/3 primary, HTTP/2 fallback, Brotli, REST primary, Gin, http3 submodule).	00-master §3 D6; additions_synthesis.md §5 C4
Zero system corruption / verify-before-apply — transport must preserve byte-identity of signed artifacts.	00-master §1 hard guarantees; additions_synthesis.md §3
Scalability single board → millions of devices — favors connection-efficient transport and small control-plane payloads.	00-master §1
update_engine streaming requires ZIP_STORED + HTTP Range so it can range-fetch payload.bin by offset/length.	aosp-update-engine §6, §7, item 2
Download resume is a first-class update_engine concern (the DISABLE_DOWNLOAD_RESUME header <i>suggests</i> resume may be the default, but this is UNVERIFIED).	aosp-update-engine §7 / Open-item 1
Catalogue-first reuse: use http3, middleware, Storage bricks rather than hand-rolling.	00-master §10 (§11.4.74)
Poll cadence 15 min + jitter (configurable) → many short-lived connections, amplifying HTTP/3 handshake/migration benefits.	00-master §2 D7; additions_synthesis.md §5 C3

3. Options considered

3.1 Transport protocol

Option A — HTTP/3 (QUIC) primary with automatic HTTP/2 fallback (CHOSEN; mandated). HTTP/3 over QUIC eliminates TCP head-of-line blocking and supports connection migration across network changes — directly relevant to mobile/field devices polling every 15 min + jitter [00-master §2 D7]. Delivered via the `vasic-digital/http3` submodule as a drop-in `net/http.Handler` with automatic fallback to HTTP/2 + standard compression [00-master §3]. The fallback is essential because many corporate/middlebox networks block UDP/443; QUIC must degrade gracefully. **Evidence caveat:** Range-request semantics over HTTP/2/3 “should hold but is unverified for Helix’s exact stack” — flagged for a spike [rauc §5].

Option B — HTTP/2 only (no QUIC). Simpler, universally reachable, no UDP/443 concern. **Rejected:** contradicts the mandated stack (D6) [00-master §3]; forfeits QUIC connection-migration and HoL-blocking benefits that the scalability + mobile-poll profile rewards [00-master §1, §2 D7].

Option C — gRPC as primary transport. Proposed by both operator drafts [additions_synthesis.md §2]. **Rejected:** mandated stack makes REST the primary/compatibility surface; gRPC is optional/internal only [additions_synthesis.md §5 C4]. gRPC also complicates the dumb-server artifact-serving path that just needs HTTPS + Range [aosp-update-engine §7].

Option D — custom server protocol (engine-defined). Several embedded engines demand their own protocol; the corpus explicitly contrasts this against Helix’s plain-HTTP device-pull model and finds plain HTTP Range “fits a custom Go control plane far better” [rauc §5]. hawkBit’s DDI is “plain HTTPS poll + token + feedback” and trivially implementable [ota_landscape_report §2.3 eclipse-hawkbit]; OSTree serves an “archive repo over plain HTTP” [ota_landscape_report §2.3 ostree]. **Rejected:** no benefit over plain HTTP(S); the wrapped engine (decided in ADR-0001, not here) sits behind the Go control plane and never dictates the public transport.

3.2 Content compression

Option A — Brotli for control-plane responses, negotiating to gzip; NO content-compression on the artifact path (CHOSEN). Brotli is mandated for content compression with negotiated fallback to gzip for older clients [00-master §3]. Applied where it pays: REST/JSON control-plane traffic (poll responses, rollout assignment payloads, telemetry, dashboard). **Critically, content compression MUST NOT be applied to the OTA artifact:** `payload.bin` is the output of `update_engine`’s `delta_generator` and is already compressed internally (the Android COW format uses gzip/XOR block compression) [aosp-update-engine §4, §5]; re-compressing it with Brotli/gzip yields negligible size reduction while **breaking the ZIP_STORED + byte-range contract** that streaming `applyPayload` depends on [aosp-update-engine §6, §7]. The artifact must be served byte-identical so its `FILE_HASH/METADATA_HASH` verify and its AOSP/payload signature stays intact [aosp-update-engine §7; android-update-engine-api.md §6] (the generic

SHA-256 + signature integrity requirement is corroborated by additions_synthesis.md §3, but the named header properties trace to the AOSP notes). This is the load-bearing rule of this ADR.

Option B — Brotli everywhere including artifacts. Rejected: transfer-encoding compression over a range-fetched, already-compressed, hash-verified blob is both useless (no size win) and harmful (defeats ZIP_STORED offset/length range fetch and risks altering bytes the device hashes) [aosp-update-engine §6, §7].

Option C — gzip-only (no Brotli). Rejected: contradicts the mandated stack (D6); Brotli is the primary, gzip the fallback [00-master §3].

Note on delta/adaptive savings: real download-size reduction for Helix comes from **incremental/delta payloads** (diffs vs PREVIOUS-target-files.zip), not transport compression [aosp-update-engine §5]. RAUC similarly distinguishes adaptive (skip-unchanged-blocks) and delta updates from transport [rauc §5]. Delta strategy is **out of scope here** and routed to **ADR-0005** [additions_synthesis.md §7].

3.3 Artifact download & resumability

Option A — device-pull streaming over HTTPS with mandatory HTTP Range; rely on update_engine's native resume (CHOSEN). The Go control plane returns {url, offset, size, FILE_HASH, FILE_SIZE, METADATA_HASH, METADATA_SIZE} and the device's update_engine byte-range-fetches payload.bin directly [aosp-update-engine §6, §7]. The object store / control plane **must** support HTTP Range Requests and serve OTA ZIP entries uncompressed (ZIP_STORED) [aosp-update-engine §7 item 2]. Resumability is **likely provided on-device**: the existence of a DISABLE_DOWNLOAD_RESUME header *suggests* download resume may be the default update_engine behaviour, but this is **UNVERIFIED** [aosp-update-engine Open-item 1] — Helix should avoid sending that header and ensure the server honours Range/conditional requests so interrupted downloads can resume rather than restart. The client must also survive process death and reconcile daemon state on rebind (re-issue applyPayload) [aosp-update-engine §6 state management]. This matches RAUC's independent finding that streaming installs over HTTP Range fit a custom Go control plane well [rauc §5].

Option B — full-file download to local storage, then file:// apply. The reconciliation flags that file:// (local-verified-file) apply vs HTTPS streaming are two distinct paths and “local-verified-file apply is safer for our verify-before-apply requirement” [additions_synthesis.md §6]. **Decision:** keep both apply paths available but make **streaming-over-Range the primary transport design** (mandated stack + scalability) while preserving the option to stage-then-verify-then-file://-apply on constrained/flaky networks; the verify-before-apply guarantee is satisfied either way because update_engine verifies FILE_HASH/METADATA_HASH + AVB/dm-verity before commit [aosp-update-engine §7, §lessons]. The streaming-vs-staged choice is a client-policy knob, not a transport-protocol change.

Option C — server-side resumable chunk protocol (e.g., casync chunk store). RAUC supports casync chunked delivery but it “needs a separate server-side chunk store” and enabling streaming means RAUC “can no longer download plain casync bundles” [rauc §5]. **Rejected for Android phase:** unnecessary given native update_engine range-fetch + resume; revisit only in the Linux phase if an engine requires it.

4. Decision

Adopt the mandated transport, composed as follows:

1. **Transport protocol: HTTP/3 (QUIC) primary** via the `vasic-digital/http3` submodule (drop-in `net/http.Handler`), with **automatic negotiation/fallback to HTTP/2** (and TLS 1.3 throughout [00-master §6 Security]) for clients/networks that cannot use QUIC/UDP-443 [00-master §3; D6]. **REST / api/v1 is the primary surface**; gRPC remains optional/internal only [additions_synthesis.md §5 C4].
2. **Content compression — two-class rule:**
 - Control-plane REST/JSON: **Brotli**, negotiating to **gzip** for older clients via `Accept-Encoding` [00-master §3].
 - OTA artifacts (`payload.bin` / OTA ZIP): **no content compression**; served **byte-identical** and **ZIP_STORED** so streaming range-fetch and hash/signature verification hold [aosp-update-engine §6, §7].
3. **Artifact delivery: device-pull streaming over HTTPS with mandatory HTTP Range Request support** on the control plane / object store (Storage brick + MinIO/S3); the control plane returns {`url`, `offset`, `size`, `FILE_HASH`, `FILE_SIZE`, `METADATA_HASH`, `METADATA_SIZE`} to the device [aosp-update-engine §7].
4. **Resumability:** do not send `DISABLE_DOWNLOAD_RESUME`, so that `update_engine`'s native download-resume behaviour (if it is indeed enabled by default — **UNVERIFIED**, see §6) is not suppressed; regardless, the server **MUST** honour Range / conditional requests so interrupted transfers can resume from offset rather than restart [aosp-update-engine Open-item 1]. Client survives process death and reconciles daemon state on rebind [aosp-update-engine §6].
5. **Apply-path policy:** streaming-over-Range is the primary path; a **stage-then-verify-then-file://-apply** fallback is retained for flaky/constrained networks. Both satisfy verify-before-apply because `update_engine` verifies hashes + AVB before commit [aosp-update-engine §7; additions_synthesis.md §6].
6. **Reuse:** compose from the verified catalogue — `http3`, `middleware`, `Storage`, `ratelimiter`, `observability` bricks — rather than hand-rolling transport/compression [00-master §10, §11.4.74].

Negotiation order at the edge: try HTTP/3 (QUIC/UDP-443) → fall back to HTTP/2 (TCP/TLS-1.3) → content-encoding `br` → `gzip` → identity. The artifact path is pinned to identity content-encoding with Range enabled regardless of the control-plane negotiation.

5. Consequences

5.1 Positive

- **Honors the locked mandated stack exactly** (D6) and resolves contradiction C4 in favour of REST-primary + HTTP/3→HTTP/2 + Brotli [additions_synthesis.md §5 C4].
- **QUIC connection migration + HoL-blocking elimination** suit mobile/field devices and the 15-min-jitter poll profile, aiding the single-board→millions scalability guarantee [00-master §1, §2 D7].

- **Two-class compression preserves byte-identity** of signed artifacts, protecting the zero-corruption / verify-before-apply guarantees [00-master §1; aosp-update-engine §7].
- **Mandatory HTTP Range + native resume** gives robust downloads on flaky networks for free, since the safety-critical apply/resume path lives in battle-tested AOSP code, not Helix [aosp-update-engine §6, §7; rauc §5].
- **Plain-HTTP device-pull** keeps the server “dumb,” matching AOSP’s no-mandated-protocol model and avoiding lock-in to any engine’s bespoke protocol [aosp-update-engine §7; rauc §5].
- **Catalogue reuse** (http3, middleware, Storage) minimizes new transport code [00-master §10].

5.2 Negative / costs

- **QUIC/UDP-443 reachability:** middlebox/corporate networks may block UDP, forcing HTTP/2 fallback; the fallback path must be continuously tested, not assumed. **UNVERIFIED (new, needs spike):** real-world QUIC reachability for the target device fleet.
- **Range semantics over HTTP/2/3 unverified for Helix’s exact stack** — must be spike-validated end-to-end (update_engine range-fetch through the http3 submodule + MinIO/S3) [rauc §5].
- **Operational discipline required** to ensure the artifact path is never accidentally Brotli/gzip-compressed or re-zipped with compression (would silently break streaming + hashing) [aosp-update-engine §6, §7]. Needs a CI/serving guard.
- **Two transport-handling code paths** (compressed control-plane vs identity artifact) add edge-config complexity.
- **ZIP_STORED packaging** is larger on disk in object storage than a compressed ZIP would be (acceptable; real savings come from delta payloads — ADR-0005) [aosp-update-engine §5].
- **update_engine resume + exact optional-header set** (incl. DISABLE_DOWNLOAD_RESUME, SWITCH_SLOT_ON_REBOOT, RUN_POST_INSTALL) is **UNVERIFIED** against the Android 15 IUpdateEngine / payload_properties.cc and must be confirmed [aosp-update-engine Open-item 1].

6. Open / UNVERIFIED items to close

1. **Range-request semantics over HTTP/3/HTTP/2 via the http3 submodule + MinIO/S3** — spike: stream a real ZIP_STORED payload.bin end-to-end and confirm byte-range correctness and resume. [rauc §5; aosp-update-engine §7]
2. **Exact update_engine resume + optional-header contract on Android 15** (DISABLE_DOWNLOAD_RESUME, SWITCH_SLOT_ON_REBOOT, RUN_POST_INSTALL) — confirm against IUpdateEngine/payload_properties.cc. [aosp-update-engine Open-item 1, §7 item 3]
UNVERIFIED
3. **QUIC/UDP-443 reachability** across the target deployment networks and the HTTP/2 fallback trigger behaviour. **UNVERIFIED (new, needs spike)**
4. **Brotli quality/parameter tuning** for control-plane JSON (CPU vs ratio) at fleet scale — benchmark; no figure asserted here. **UNVERIFIED (new)**

5. **Interaction of any future RAUC/Linux-phase NBD/Range streaming with Helix Brotli/HTTP3** [ota_landscape_report §5 item 6; rauc §5] — deferred to the Linux phase.
6. **Delta/adaptive download-size strategy** (the real bandwidth lever) — routed to **ADR-0005**, not decided here. [additions_synthesis.md §7]

7. Status

Proposed. Pending operator review gate alongside ADR-0001..0005. The mandated stack elements (HTTP/3→HTTP/2, Brotli, REST-primary) are locked (D6) and not re-litigated; this ADR's *composition* choices (two-class compression, mandatory Range, native-resume policy, streaming-vs-staged apply) become binding on approval and are gated on closing the §6 spike items before the corresponding code paths are marked stable.

8. Compliance notes (HelixConstitution)

Clause	How this ADR complies
§11.4.6 (no guessing)	Every transport claim cites a source note or the master design; unverifiable items (QUIC reachability, Range-over-HTTP3, Android-15 resume headers, Brotli tuning) are carried as UNVERIFIED rather than asserted. [aosp-update-engine; rauc; 00-master]
§11.4.8 (research before implementation)	This is an evidence-based ADR derived from the landscape report + stack notes + additions synthesis; binding code paths gated behind the §6 spikes before being marked stable. [00-master §16]
§11.4.74 (catalogue-first reuse)	Transport/compression composed from verified bricks (http3, middleware, Storage, ratelimiter, observability); no hand-rolled QUIC/Brotli stack. [00-master §10]
§1 (four-layer testing) / §7.1 (no-bluff)	§6 spike items are written as runtime/integration validations (stream a real ZIP_STORED payload, exercise fallback, confirm resume) producing positive evidence; safety-critical serving correctness (byte-identity, Range, hash-verify) is treated as a ≥90% path. [00-master §13]
§11.4.123 (rock-solid proof)	The §6 spikes are the rock-solid-proof mechanism for this ADR: each UNVERIFIED claim (QUIC reachability, Range-over-HTTP3, Android-15 resume/optional-header contract, Brotli tuning) is closed by an

Clause	How this ADR complies
D2 / D6 (locked decisions honored)	executable spike that produces positive runtime evidence before the dependent code path is marked stable, rather than by assertion. [00-master §13, §16] Native A/B + custom Go control plane retained; mandated stack (HTTP/3→HTTP/2, Brotli, REST-primary, Gin, http3 submodule) adopted verbatim; wrap-engine choice left to ADR-0001 (not re-opened). [00-master §2]
§1 zero-corruption guarantee	Two-class compression rule keeps signed artifacts byte-identical; verify-before-apply preserved via update_engine hash/AVB checks on both streaming and file:// paths. [00-master §1; aosp-update-engine §7]